Banco de Questões Unificado

DevEvolution

ASSUNTO 1: Linhas de Código (LOC)

Questão 1 — Conceitual (1 Ponto)
Pergunta: O que a métrica LOC mede diretamente em um software?
[ ] A) A complexidade dos algoritmos.
[x] B) O tamanho do código-fonte. (CORRETA)
[ ] C) A velocidade de execução.
[ ] D) A quantidade de bugs encontrados.
Explicação (Flashcard): LOC (Lines of Code) mede o tamanho do software contando suas linhas de código-fonte.

Questão 2 — Vantagens e Desvantagens (2 Pontos)
Pergunta: Qual é a principal limitação técnica da métrica LOC?
[ ] A) É muito difícil de ser automatizada.
[ ] B) Exige cálculos matemáticos complexos.
[x] C) Mede apenas quantidade, não qualidade ou complexidade. (CORRETA)
[ ] D) Só funciona para a linguagem Java.
Explicação (Flashcard): A métrica LOC mede quantidade, não qualidade. Ela falha em capturar a complexidade real ou a qualidade do software.

Questão 3 — Exemplo Prático de Código (3 Pontos)
Pergunta: Analise o código:

```
if (x > 0) {
    y = x + 1;
}
```

Quantas linhas físicas e lógicas existem, respectivamente?
[ ] A) 1 linha física e 3 linhas lógicas.
[x] B) 3 linhas físicas e 1 linha lógica. (CORRETA)
[ ] C) 3 linhas físicas e 3 linhas lógicas.
[ ] D) 2 linhas físicas e 2 linhas lógicas.
Explicação (Flashcard): Linhas físicas contam cada linha do arquivo (3 linhas textuais). Linhas lógicas contam apenas as instruções reais executadas (1 comando condicional).

Questão 4 — DESAFIO (4 PONTOS)
Pergunta: Se o desenvolvedor reduzir um método verboso de 18 LOC para um método limpo de 7 LOC, mantendo o mesmo comportamento, qual o principal impacto técnico?
[ ] A) O programa executará mais lentamente.
[x] B) Reduz o esforço de manutenção futura do código. (CORRETA)
[ ] C) O código perderá funções importantes para o usuário.
[ ] D) Aumenta a probabilidade de erros ocultos.
Explicação (Flashcard): Códigos menores e mais limpos facilitam a legibilidade, reduzindo o custo de manutenção futura e o risco de bugs.

ASSUNTO 2: Pontos de Função (PF)

Questão 1 — Conceitual (1 Ponto)
Pergunta: Qual é a perspectiva de medição utilizada pelos Pontos de Função (PF)?
[ ] A) A visão puramente técnica do programador.
[x] B) A visão do usuário baseada em funcionalidades. (CORRETA)
[ ] C) O número total de linhas do banco de dados.
[ ] D) O número de cliques na tela do sistema.
Explicação (Flashcard): Os Pontos de Função medem o tamanho do software com base nas funcionalidades entregues, focando na visão do usuário (o que o sistema faz).

Questão 2 — Vantagens e Desvantagens (2 Pontos)
Pergunta: Qual é a maior vantagem dos Pontos de Função em relação ao LOC?
[ ] A) É muito mais simples e rápida de calcular.
[ ] B) Não exige nenhum tipo de conhecimento especializado.
[x] C) É independente da linguagem de programação usada. (CORRETA)
[ ] D) Garante que o código gerado estará livre de erros.
Explicação (Flashcard): Pontos de Função são totalmente independentes da linguagem de programação e podem ser calculados antes mesmo de o código ser escrito.

Questão 3 — Exemplo Prático de Código (3 Pontos)
Pergunta: Um método lê dados digitados pelo usuário, salva em um banco de dados interno e exibe uma confirmação. Quais elementos de PF estão presentes?
[x] A) 1 Entrada (EI), 1 Arquivo Interno (ILF) e 1 Saída (EO). (CORRETA)
[ ] B) 3 Consultas Externas (EQ).
[ ] C) 2 Interfaces Externas (EIF) e 1 Saída (EO).
[ ] D) 3 Entradas Externas (EI) apenas.
Explicação (Flashcard): Os dados inseridos são Entrada Externa (EI), o armazenamento é Arquivo Lógico Interno (ILF) e a confirmação é uma Saída Externa (EO).

Questão 4 — DESAFIO (4 PONTOS)
Pergunta: Considerando os pesos padrões (EI=4, EO=5, ILF=10), calcule o total de PF de uma aplicação com 2 telas de cadastro (EI), 1 relatório (EO) e 1 tabela interna (ILF).
[ ] A) 19 PF.
[x] B) 23 PF. (CORRETA)
[ ] C) 32 PF.
[ ] D) 40 PF.
Explicação (Flashcard): Aplicando a fórmula:

$$\text{Total PF} = (2 \times 4) + (1 \times 5) + (1 \times 10) = 8 + 5 + 10 = 23 \text{ PF}.$$

ASSUNTO 3: Complexidade Ciclomática (McCabe)

Questão 1 — Conceitual (1 Ponto)
Pergunta: O que a Complexidade Ciclomática de McCabe mede diretamente?
[ ] A) A quantidade de arquivos criados no projeto.
[x] B) O número de caminhos independentes no código. (CORRETA)
[ ] C) A velocidade com que a equipe escreve o código.
[ ] D) O número de comentários inseridos nas funções.
Explicação (Flashcard): A Complexidade Ciclomática mede o número de caminhos linearmente independentes de execução que o código pode seguir.

Questão 2 — Vantagens e Desvantagens (2 Pontos)
Pergunta: Qual é a utilidade prática do valor da Complexidade Ciclomática para a equipe de testes?
[x] B) Indica o número mínimo de testes unitários necessários. (CORRETA)
[ ] A) Define o tempo máximo de duração de um teste de carga.
[ ] C) Substitui a necessidade de realizar testes manuais.
[ ] D) Indica quantos bugs existem de forma exata na função.
Explicação (Flashcard): O valor da complexidade determina o número mínimo de testes unitários necessários para garantir 100% de cobertura de caminhos.

Questão 3 — Exemplo Prático de Código (3 Pontos)
Pergunta: Qual é a Complexidade Ciclomática do método abaixo usando a fórmula $V(G) = P + 1$?

```
function verificar(x) {
    if (x > 10) {
        return "Maior";
    }
    return "Menor";
}
```

[ ] A) 1.
[x] B) 2. (CORRETA)
[ ] C) 3.
[ ] D) 4.
Explicação (Flashcard): O código possui apenas 1 ponto de decisão ou nó-predicado (if). Aplicando a fórmula $V(G) = P + 1 = 1 + 1 = 2$.

Questão 4 — DESAFIO (4 PONTOS)
Pergunta: Se um grafo de fluxo de controle de um método possui 10 arestas (E) e 8 nós (N), qual é a sua complexidade ciclomática usando a fórmula $V(G) = E - N + 2$?
[ ] A) 2.
[x] B) 4. (CORRETA)
[ ] C) 6.
[ ] D) 10.
Explicação (Flashcard): Aplicando a fórmula estrutural do grafo:

$$V(G) = E - N + 2 = 10 - 8 + 2 = 4.$$

ASSUNTO 4: Profundidade de Aninhamento (Nesting Depth)

Questão 1 — Conceitual (1 Ponto)
Pergunta: O que a métrica Profundidade de Aninhamento (Nesting Depth) avalia?
[ ] A) Quantos arquivos estão dentro de uma mesma pasta.
[x] B) O número de níveis de blocos de código um dentro do outro. (CORRETA)
[ ] C) A quantidade de classes herdadas no arquivo.
[ ] D) A profundidade do banco de dados relacional.
Explicação (Flashcard): A profundidade de aninhamento mede o número de níveis de blocos de código que estão inseridos ou colocados um dentro do outro.

Questão 2 — Vantagens e Desvantagens (2 Pontos)
Pergunta: Por que manter uma profundidade de aninhamento baixa é considerado uma boa

prática de design?
[x] A) Torna o código muito mais legível e reduz riscos de erros. (CORRETA)
[ ] B) Faz o código rodar com metade do uso de memória.
[ ] C) Evita que o desenvolvedor precise documentar o sistema.
[ ] D) Permite que o código seja compilado mais rapidamente.
Explicação (Flashcard): Menor profundidade está diretamente ligada à qualidade e legibilidade do código, facilitando a manutenção e reduzindo riscos de erros.

Questão 3 — Exemplo Prático de Código (3 Pontos)
Pergunta: Qual a profundidade máxima de aninhamento do trecho de código abaixo?

```
for (let i = 0; i < 10; i++) {
    if (i > 5) {
        console.log(i);
    }
}
```

[ ] A) 1.
[x] B) 2. (CORRETA)
[ ] C) 3.
[ ] D) 4.
Explicação (Flashcard): Há um bloco condicional if (Nível 2) inserido dentro de um laço de repetição for (Nível 1). A profundidade máxima é 2.

Questão 4 — DESAFIO (4 PONTOS)
Pergunta: Um método possui uma estrutura com 5 ifs aninhados (um dentro do outro). Qual a melhor prática de refatoração para corrigir esse code smell de alta profundidade?
[ ] A) Juntar todas as condicionais em uma única linha usando operadores lógicos complexos.
[x] B) Aplicar a extração de métodos ou cláusulas de guarda (guard clauses). (CORRETA)
[ ] C) Mudar a linguagem de programação do projeto.
[ ] D) Reescrever a lógica substituindo os blocos por laços de repetição do tipo for.
Explicação (Flashcard): Para reduzir a profundidade, aplicamos técnicas de refatoração como simplificação de condicionais e extração de blocos para novos métodos.

ASSUNTO 5: Quantidade de Parâmetros por Método

Questão 1 — Conceitual (1 Ponto)
Pergunta: O que a métrica de Quantidade de Parâmetros analisa em um método?
[ ] A) A quantidade de variáveis locais criadas dentro da função.
[ ] B) O número de vezes que a função é chamada pelo sistema.
[x] C) O número de parâmetros formais presentes na assinatura. (CORRETA)
[ ] D) O tamanho das strings enviadas como argumento.
Explicação (Flashcard): Ela conta o número de parâmetros formais na assinatura do método. Mais parâmetros geram maior complexidade cognitiva.

Questão 2 — Vantagens e Desvantagens (2 Pontos)
Pergunta: De acordo com as boas práticas de qualidade, a partir de quantos parâmetros um método é considerado de alta complexidade (Code Smell)?
[ ] A) 2 parâmetros.
[ ] B) 3 parâmetros.
[ ] C) 4 parâmetros.
[x] D) 5 ou mais parâmetros. (CORRETA)

Explicação (Flashcard): Funções com 0 a 2 parâmetros são simples; 3 a 4 são moderadas; e 5 ou mais configuram alta complexidade (code smell).

Questão 3 — Exemplo Prático de Código (3 Pontos)
Pergunta: Analise as assinaturas de funções abaixo:
1. `function criar(nome, email)`
2. `function enviar(id, item, preco, cupom, frete)`
Qual a classificação de complexidade delas, respectivamente?
[ ] A) Ambas são de baixa complexidade.
[x] B) 1 é baixa complexidade e 2 é alta complexidade (Code Smell). (CORRETA)
[ ] C) 1 é alta complexidade e 2 é baixa complexidade.
[ ] D) Ambas possuem complexidade moderada.
Explicação (Flashcard): A assinatura 1 possui 2 parâmetros (baixa complexidade). A assinatura 2 possui 5 parâmetros, configurando um Code Smell de alta complexidade.

Questão 4 — DESAFIO (4 PONTOS)
Pergunta: Qual refatoração é recomendada quando um método precisa receber mais de 5 parâmetros para realizar uma operação?
[ ] A) Passar todos os dados concatenados em uma única string gigante.
[ ] B) Utilizar variáveis globais para evitar passar os dados por parâmetro.
[x] C) Agrupar os parâmetros relacionados em um único objeto de configuração. (CORRETA)
[ ] D) Duplicar o método para que cada cópia receba metade dos parâmetros.
Explicação (Flashcard): Para reduzir parâmetros excessivos, agrupamos os dados em uma estrutura de objeto (padrão Parameter Object).

ASSUNTO 6: Índice de Manutenibilidade (MI)

Questão 1 — Conceitual (1 Ponto)
Pergunta: O que é o Índice de Manutenibilidade (MI)?
[ ] A) Uma contagem do número de commits de correção feitos no repositório.
[ ] B) O tempo médio que o servidor leva para reiniciar após uma falha.
[x] C) Uma métrica composta que avalia a facilidade de manter o código. (CORRETA)
[ ] D) O número de desenvolvedores focados em suporte na equipe.
Explicação (Flashcard): MI (Maintainability Index) é uma métrica composta que combina múltiplos fatores estruturais para resumir a facilidade de manter o software.

Questão 2 — Vantagens e Desvantagens (2 Pontos)
Pergunta: Como se interpreta o score numérico retornado pelo cálculo do MI?
[x] A) Valores altos indicam código simples e fácil de manter. (CORRETA)
[ ] B) Valores baixos significam que o código está muito bem projetado.
[ ] C) Valores altos representam alto risco de falha na produção.
[ ] D) O score não tem relação direta com a qualidade do software.
Explicação (Flashcard): Na interpretação do score do MI, quanto mais alto o valor, mais fácil é a manutenção. Valores baixos indicam risco.

Questão 3 — Exemplo Prático de Código (3 Pontos)
Pergunta: Analise os dados coletados por uma ferramenta estática:
Módulo A: MI = 115 (Verde)
Módulo B: MI = 32 (Vermelho)
Qual deve ser a atitude técnica da equipe?
[ ] A) Deletar o Módulo A por segurança.

[x] B) Priorizar a refatoração urgente do Módulo B. (CORRETA)
[ ] C) Reescrever o Módulo A em outra linguagem de programação.
[ ] D) Ignorar o relatório, pois o sistema está funcionando em produção.
Explicação (Flashcard): Um MI baixo (Vermelho) indica que aquele trecho do código é muito difícil de manter e evoluir, exigindo refatoração imediata.

Questão 4 — DESAFIO (4 PONTOS)
Pergunta: O cálculo matemático do Índice de Manutenibilidade (MI) combina tradicionalmente quais pilares do código-fonte?
[ ] A) Custo financeiro, prazo do projeto e esforço do desenvolvedor.
[ ] B) Número de classes, número de requisitos e banco de dados.
[x] C) Linhas de Código (LOC), Complexidade Ciclomática e Métricas de Halstead. (CORRETA)
[ ] D) Tempo de compilação, taxa de defeitos e quantidade de testes unitários.
Explicação (Flashcard): O MI é calculado de forma automatizada combinando o tamanho (LOC), a complexidade estrutural (Ciclomática) e as métricas de volume de Halstead.

ASSUNTO 7: Code Smells (Heurísticas de Design)

Questão 1 — Conceitual (1 Ponto)
Pergunta: O que são Code Smells no contexto da engenharia de software?
[ ] A) Erros graves de sintaxe que impedem a compilação do programa.
[x] B) Padrões estruturais no código que sugerem problemas de design. (CORRETA)
[ ] C) Falhas de segurança exploradas por invasores externos.
[ ] D) Códigos de teste que foram esquecidos no ambiente de produção.
Explicação (Flashcard): Code Smells não são bugs ou erros, mas sim padrões no código que sugerem problemas de design e degradação de manutenibilidade.

Questão 2 — Vantagens e Desvantagens (2 Pontos)
Pergunta: Qual é a principal desvantagem de ignorar a presença de Code Smells no código-fonte ao longo do tempo?
[ ] A) O sistema deixará de compilar no dia seguinte.
[x] B) Aumenta drasticamente o esforço e o custo de manutenção futura (Dívida Técnica). (CORRETA)
[ ] C) A equipe será impedida de subir o sistema para produção.
[ ] D) O banco de dados perderá os dados cadastrados.
Explicação (Flashcard): Ignorar esses sinais aumenta o esforço de compreensão e evolução do código, acumulando dívida técnica.

Questão 3 — Exemplo Prático de Código (3 Pontos)
Pergunta: Uma classe chamada `GerenciadorGeral` possui 2500 linhas de código e é responsável por ler arquivos, processar pagamentos, cadastrar usuários e enviar e-mails. Qual Code Smell clássico está ocorrendo?
[ ] A) Duplicação de Código (Duplicate Code).
[ ] B) Método Longo (Long Method).
[x] C) Classe Deus (God Class / Large Class). (CORRETA)
[ ] D) Excesso de Parâmetros (Long Parameter List).
Explicação (Flashcard): Uma classe gigante que acumula responsabilidades demais e centraliza a lógica do sistema é conhecida como God Class.

Questão 4 — DESAFIO (4 PONTOS)
Pergunta: Qual princípio fundamental do design de software orientado a objetos é violado pela

presença de uma God Class e como corrigi-lo?
[ ] A) Princípio do Aberto/Fechado (OCP), corrigido adicionando herança complexa.
[x] B) Princípio de Responsabilidade Única (SRP), corrigido dividindo a classe em componentes especializados. (CORRETA)
[ ] C) Princípio de Substituição de Liskov (LSP), corrigido com tratamento de exceções.
[ ] D) Princípio da Inversão de Dependência (DIP), corrigido excluindo interfaces.
Explicação (Flashcard): Uma God Class viola o Princípio de Responsabilidade Única (Single Responsibility Principle). A correção consiste em dividir suas funções em classes menores e focadas.

ASSUNTO 8: Indicadores de Churn (Frequência de Mudanças)

Questão 1 — Conceitual (1 Ponto)
Pergunta: O que mede a métrica de Churn de Código?
[ ] A) O tempo necessário para carregar as dependências de um projeto.
[x] B) O volume de alterações e modificações no código ao longo do tempo. (CORRETA)
[ ] C) A quantidade de novas funcionalidades solicitadas pelo cliente.
[ ] D) O número de usuários ativos que desistem de usar o sistema.
Explicação (Flashcard): O Churn mede a quantidade de modificações (linhas adicionadas, removidas ou alteradas) feitas no código ao longo do tempo.

Questão 2 — Vantagens e Desvantagens (2 Pontos)
Pergunta: O que um Churn de Código extremamente elevado em um único arquivo do sistema costuma indicar?
[x] A) Que o código daquele arquivo está instável ou os requisitos mudam frequentemente. (CORRETA)
[ ] B) Que a qualidade técnica daquele arquivo é excelente.
[ ] C) Que o arquivo foi totalmente coberto por testes automatizados.
[ ] D) Que o arquivo não sofreu nenhuma modificação recente.
Explicação (Flashcard): Churn alto indica código instável, requisitos mal definidos ou baixa previsibilidade de manutenção naquela parte do sistema.

Questão 3 — Exemplo Prático de Código (3 Pontos)
Pergunta: O painel de controle do repositório mostrou que o arquivo `pagamento.js` foi alterado 84 vezes no último mês, enquanto outros arquivos mudaram apenas uma vez. Como classificar este indicador?
[ ] A) Churn saudável e estável.
[x] B) Churn alto, indicando provável foco de instabilidade no pagamento. (CORRETA)
[ ] C) Churn nulo, indicando código que não precisa de atenção.
[ ] D) Churn de banco de dados isolado.
Explicação (Flashcard): Um arquivo alterado dezenas de vezes a mais que a média apresenta alto Churn, sinalizando instabilidade e provável necessidade de refatoração.

Questão 4 — DESAFIO (4 PONTOS)
Pergunta: Como se comporta o indicador de Churn em sistemas projetados com alta manutenibilidade e baixo acoplamento?
[ ] A) Qualquer alteração exige modificar todos os arquivos do sistema simultaneamente.
[ ] B) O Churn se torna nulo e o código nunca mais é alterado.
[x] C) As modificações são localizadas, seguras e afetam poucos arquivos específicos (Churn localizado). (CORRETA)
[ ] D) O Churn aumenta de forma exponencial a cada nova entrega.

Explicação (Flashcard): Sistemas bem projetados e manuteníveis possuem alta coesão e baixo acoplamento, fazendo com que as alterações fiquem localizadas e restritas a pouquíssimos arquivos.

ASSUNTO 9: Número de Módulos, Classes, Métodos ou Requisitos

Questão 1 — Conceitual (1 Ponto)
Pergunta: Ao contrário da métrica LOC (linhas de código), o que as métricas como Número de Módulos, Classes ou Métodos medem diretamente?
[x] A) O tamanho do software por meio de seus componentes estruturais de alto nível. (CORRETA)
[ ] B) A quantidade de memória física que o sistema ocupa ao ser compilado.
[ ] C) O número exato de bugs encontrados durante os testes de estresse.
[ ] D) A velocidade de conexão de rede de cada módulo de comunicação.
Explicação (Flashcard): Estas métricas medem o tamanho e a divisão do produto olhando para a estrutura de classes, módulos e componentes lógicos, em vez de contar linhas de código brutas no arquivo de texto.

Questão 2 — Vantagens e Desvantagens (2 Pontos)
Pergunta: Qual é a principal vantagem estratégica de quantificar o "Número de Requisitos" logo no início do projeto?
[x] A) Estimar o escopo e o esforço de desenvolvimento antes mesmo de codificar o sistema. (CORRETA)
[ ] B) Garantir de forma automática que o código gerado estará livre de Code Smells.
[ ] C) Substituir a necessidade de se fazerem testes unitários ou de integração no sistema.
[ ] D) Reduzir o custo financeiro do projeto dividindo as tarefas sem precisar de programadores.
Explicação (Flashcard): Os requisitos definem o que o cliente quer. Contá-los no início ajuda a quantificar a complexidade do escopo e serve como uma base sólida para estimar o esforço inicial do projeto antes do código existir.

Questão 3 — Exemplo Prático de Código (3 Pontos)
Pergunta: Um relatório de análise estática indicou que o seu sistema possui 10 classes e 120 métodos no total. Qual é a média de métodos por classe e o impacto disso na manutenibilidade?
[x] A) Média de 12. Há risco de classes acumulando responsabilidades (Code Smell). (CORRETA)
[ ] B) Média de 120. Indica que a estrutura está perfeitamente saudável e limpa.
[ ] C) Média de 12. Significa uma estrutura ideal e super distribuída.
[ ] D) Média de 1,2. Indica que faltam códigos para o sistema conseguir rodar.
Explicação (Flashcard): A média é de 12 métodos por classe (120/10). Manter um número médio alto indica que as classes podem estar sobrecarregadas, acumulando responsabilidades demais, o que dificulta o entendimento e os testes.

Questão 4 — DESAFIO (4 PONTOS)
Pergunta: Uma única classe chamada `ProcessadorDePedidos` possui 45 métodos implementados. Qual refatoração ataca diretamente esse Code Smell respeitando o design de software saudável?
[x] A) Extrair novas classes com responsabilidades menores (ex: PedidoPagamento, PedidoEnvio). (CORRETA)
[ ] B) Agrupar e fundir os 45 métodos em apenas 1 método gigante de 1500 linhas de código.
[ ] C) Reescrever todo o algoritmo em outra linguagem de programação mais compacta.
[ ] D) Adicionar mais comentários ao longo da classe para justificar a presença de cada método.
Explicação (Flashcard): Se uma classe possui um número excessivo de métodos, ela acumulou

responsabilidades demais e está se transformando em um Code Smell. A solução ideal é quebrar o arquivo em classes menores e focadas (coesas), mantendo o número de métodos baixo para facilitar o entendimento e o reaproveitamento.